



Project Presentation of CO-OP Project at Industry (Module-2) (22CS421)

On

SHOUVAIR

A Clothing E-Commerce Web Application

Presented by

Aarav Shourie - 2210990010

Dhruv Kundrai - 2210990276

Aarya Rishi - 2210990012

Advitya Hans - 2210990076

Supervised By

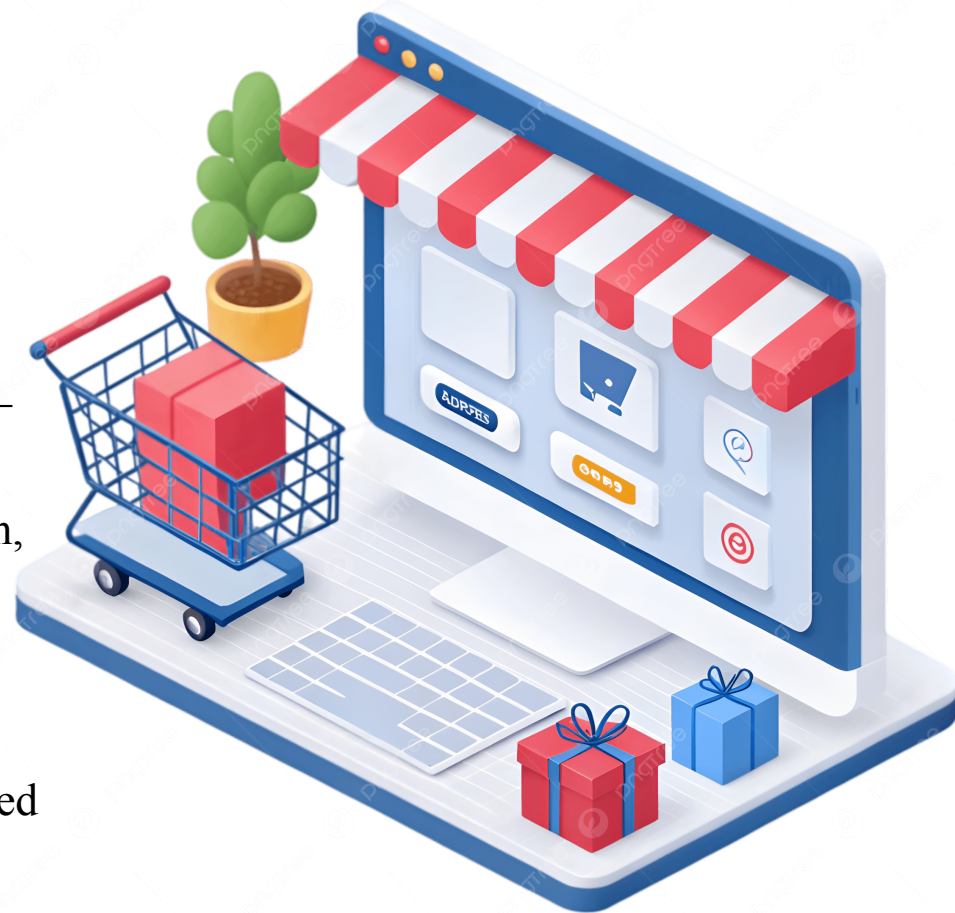
Dr. Gurpreet Singh

Department of Computer Science and Engineering,
Chitkara University, Punjab

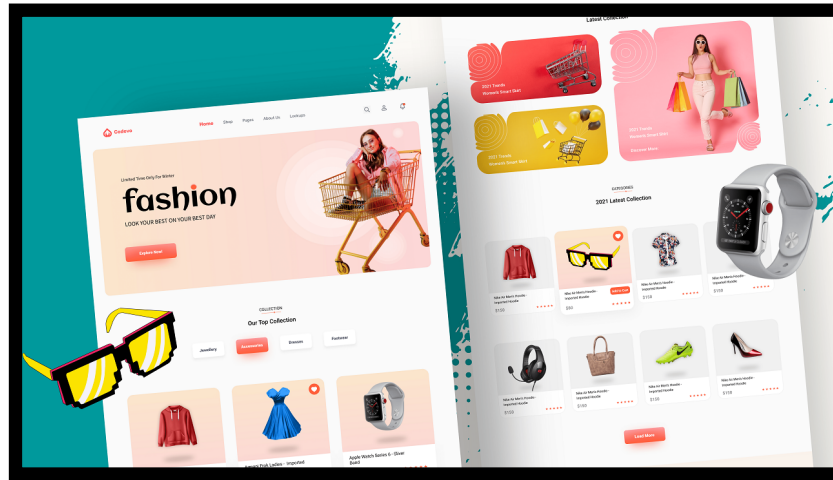
Abstract



- SHOUVAIR is a full-stack clothing e-commerce web application built on Node.js, Express.js, MongoDB, and EJS server-side templating.
- The platform offers six product categories — Shirts, Polos, Dresses, Bags, Caps, and Ties — with user registration, JWT authentication, and order history management.
- An interactive Leaflet.js v1.9.4 map embedded in checkout enables customers to pin precise delivery coordinates that are stored alongside each order.
- A three-schema MongoDB architecture (User, Order, Contact) backed by Mongoose ODM provides schema validation, type casting, and consistent document structure.
- Security is enforced through bcrypt password hashing, Helmet.js HTTP headers, CORS, HTTP-only cookie storage, and parameterized Mongoose queries.



Introduction

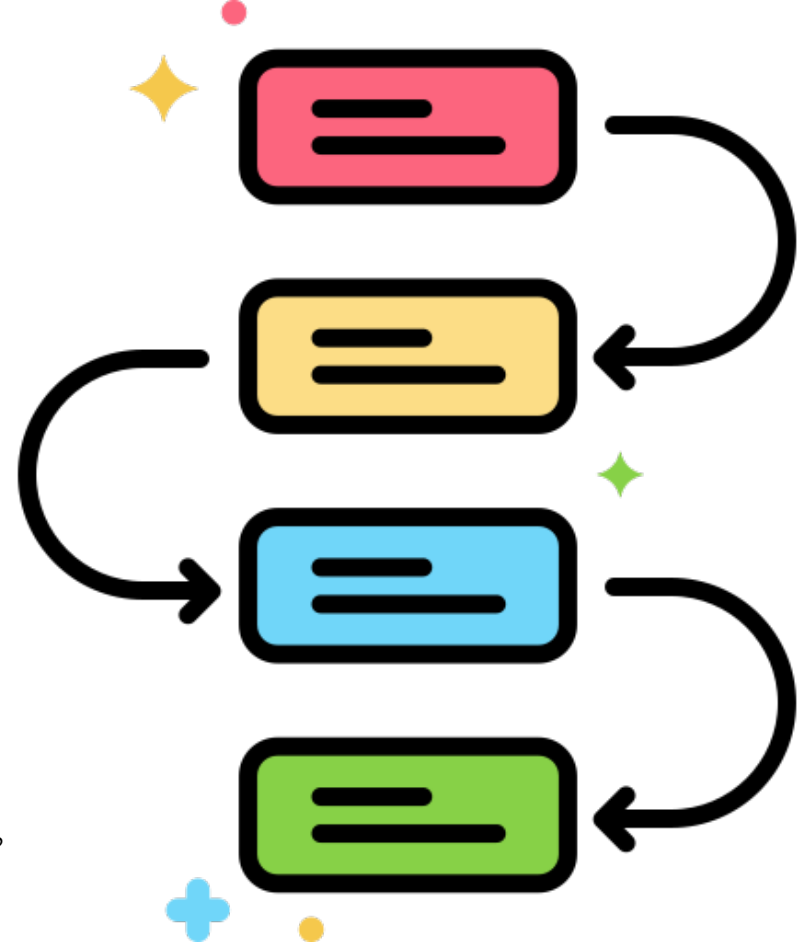


- SHOUVAIR is a multi-page online clothing storefront serving registered users across six dedicated product categories.
- Authored by Dhruv Kundrai with team members Aarya Rishi, Aarav Shourie, and Advitya Hans during 2025–2026.
- Small and mid-sized clothing brands lack affordable bespoke e-commerce solutions, often relying on generic marketplaces with high commissions and limited customization.
- The project delivers a fully ownable, brand-cohesive codebase that a small team can iterate on without microservice complexity.
- Key objectives include secure stateless authentication, server-rendered multi-page UX, interactive map-based checkout, and complete control over customer and order data.

Project Plan



- Phase 1 – Architecture & Tech Stack: Selected Node.js, Express.js, MongoDB, EJS, and JWT for a clean monolithic design.
- Phase 2 – Schema Design: Defined Mongoose schemas for User, Order, and Contact with required-field validation.
- Phase 3 – Authentication System: Built JWT cookie auth with bcrypt password hashing and HTTP-only cookie storage.
- Phase 4 – Frontend Templates: Built 13 EJS view templates with Cinzel, Playfair Display, and Poppins typography.
- Phase 5 – Map-Based Checkout: Integrated Leaflet.js v1.9.4 to capture delivery latitude and longitude on click.
- Phase 6 – Security & Testing: Added Helmet headers, CORS, env validation, and authored the /tests test suite.



Scope of the Project

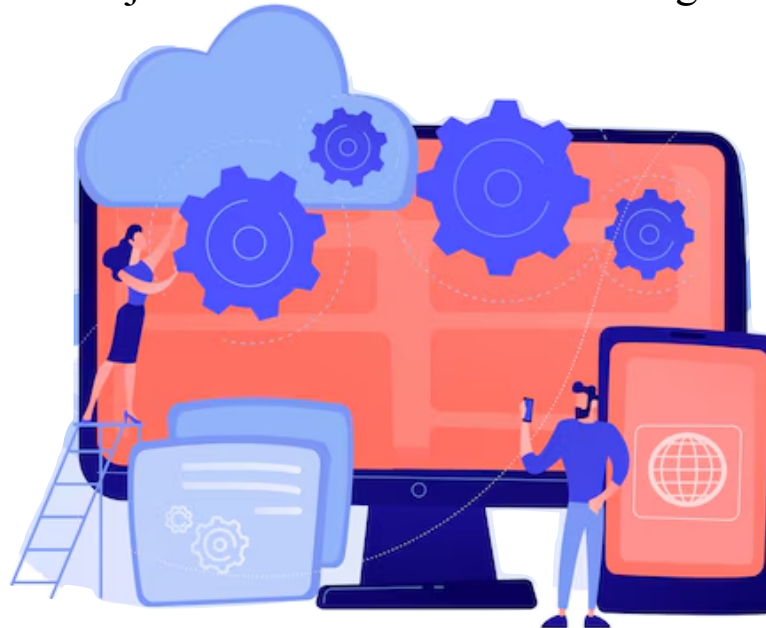


- Complete Express.js server handling authentication, product browsing, order management, and contact submissions.
- JWT-based user authentication and authorization with HTTP-only cookie management and 7-day default expiration.
- 18 source files across server logic, utilities, public static assets, and EJS view templates.
- MongoDB schemas for User, Order, and Contact entities defined and instantiated via Mongoose ODM.
- Leaflet.js v1.9.4 interactive delivery map embedded directly in the checkout (Reserve) page.
- 13 server-rendered EJS templates covering Welcome, Login, Dashboard, Reserve, Profile, About, Contact, and six product categories.
- Comprehensive test suite under the /tests directory covering authentication and data persistence.

System Requirements



- Runtime: Node.js v18.0.0 or higher on any compatible hosting environment.
- Database: MongoDB (e.g., MongoDB Atlas) accessed via Mongoose 8.14.3 ODM.
- Web Framework: Express.js v4.21.2 with EJS 3.1.10 server-side template engine.
- External Integrations: Leaflet.js v1.9.4 tile services and Google Fonts CDN (Cinzel, Playfair Display, Poppins).

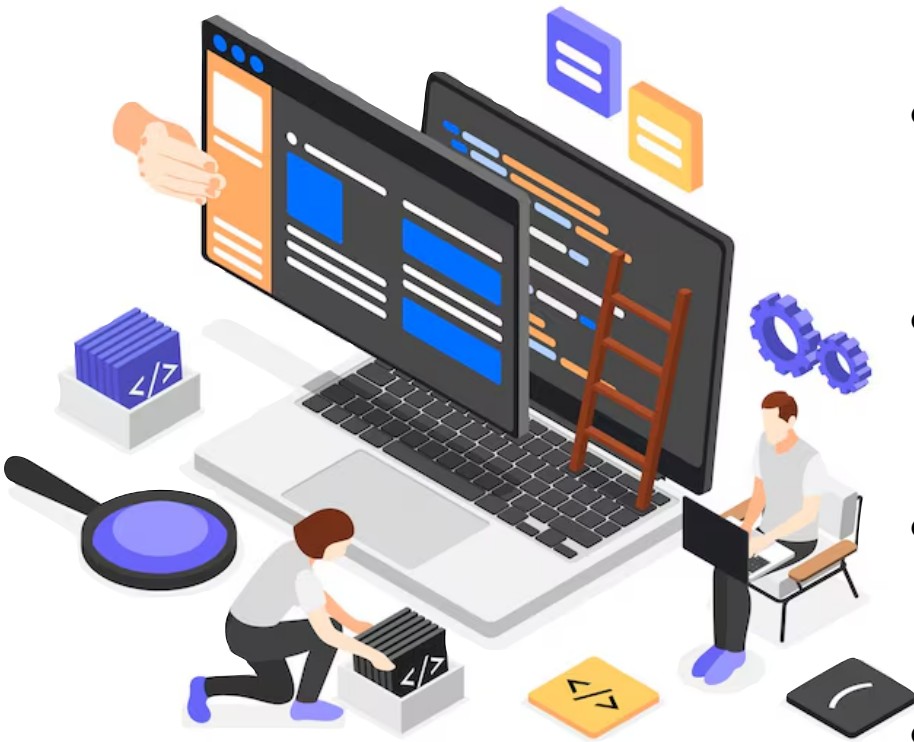


- Configuration: .env file with MONGO_URI, JWT_SECRET, JWT_EXPIRES, and PORT (default 8080).
- Production: Reverse proxy such as Nginx or Caddy recommended for TLS termination.



- Server-Side Rendering: EJS templates rendered server-side ensure complete HTML on first paint, improving perceived performance and discoverability.
- Document-Oriented Persistence: MongoDB with Mongoose ODM provides schema validation, type casting, and middleware hooks without relational migration overhead.
- Stateless JWT Authentication: HTTP-only cookies enable horizontal scaling without sticky sessions and protect tokens from XSS access.
- Map-Integrated Checkout: Leaflet.js v1.9.4 click event listeners capture geographic coordinates submitted alongside the order form.
- Defense-in-Depth Security: bcrypt password hashing, Helmet.js HTTP headers, CORS policy, and exclusive use of environment variables for secrets.





- Three-Tier Monolithic Architecture: EJS presentation layer, Express.js business logic, and MongoDB data layer accessed via Mongoose.
- Single Express server (server.js) handles all routing, authentication middleware, and view rendering through res.render().
- Three Mongoose Schemas: User stores credentials, Order stores purchase records with map coordinates, Contact stores form submissions.
- 13 EJS Templates: welcome, login, dashboard, reserve, user, about, contact, plus six product category pages (shirts, polos, dress, bag, cap, ties).
- Auth Middleware verifies JWT cookies before route handlers execute; unauthenticated requests redirected to /login rather than returning 401 JSON.
- Static Assets in /public include client-side script.js for form UX and animated panel style.css.

Implementation Details



- Authentication: bcrypt hashing with configurable salt rounds, JWT issuance via jsonwebtoken with HS256 signing and HTTP-only cookies.
- Route Protection: Middleware extracts and verifies the JWT cookie; protected pages include /dashboard, /reserve, /user, /about, and /contact.



- Reserve (Checkout) Flow: User clicks Leaflet.js map → JavaScript captures lat/ lng → POST /reserve creates an Order document in MongoDB.
- User Profile: GET /user retrieves all Order documents tied to the authenticated user and renders complete order history with stored coordinates.
- Contact Form: POST /contact validates required name, emailAddress, and yourMessage fields and persists a Contact document.
- Category Pages: Six EJS templates (shirts.ejs, polos.ejs, dress.ejs, bag.ejs, cap.ejs, ties.ejs) provide focused per-category browsing.



- Authentication Tests: registration, login, JWT cookie issuance, and verification flows under the /tests directory.
- Route Protection Tests confirm unauthenticated requests are correctly redirected to the login page.
- Mongoose Schema Validation: required-field enforcement and type checking on User, Order, and Contact schemas.
- Order Creation Tests: full checkout flow from POST /reserve through MongoDB persistence including map coordinates.
- Contact Form Tests: POST /contact data validation and storage of Contact documents.
- Code Quality: structured try-catch on all async database operations, Morgan request logging, and startup environment-variable validation.

Conclusion & Future Scope

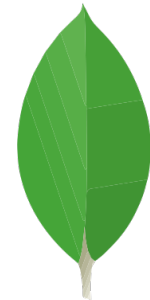


- Delivered a complete, production-ready full-stack e-commerce web app with 18 source files and 13 original EJS view templates.
- Filed for copyright registration on 25 April 2026 (Diary No. SW-19635/2026-CO, Receipt No. 239849) under the Copyright Act, 1957.
- Future Scope: Razorpay or Stripe payment gateway integration and an admin dashboard for inventory and order fulfilment.
- Feature Expansion: product search and filters, email notifications, wishlist, ratings & reviews, and a coupon/discount engine.
- Scalability Roadmap: microservices migration for auth/catalogue/orders, Redis caching, and CDN for product images.
- Frontend Evolution: potential React or Next.js migration for richer client-side interactivity while retaining the MongoDB backend.

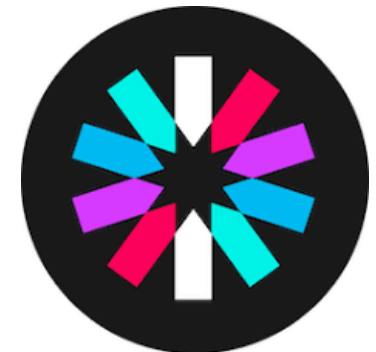


Tools & Technologies

- Backend: Node.js v18+ with Express.js 4.21.2 for routing and middleware.
- Database: MongoDB with Mongoose 8.14.3 ODM for schema validation and document operations.
- View Engine: EJS 3.1.10 for server-side HTML template rendering.
- Auth & Security: jsonwebtoken 9.0.2, bcrypt 6.0.0, Helmet 8.0.0, cookie-parser 1.4.7, and CORS 2.8.5.
- Maps & Typography: Leaflet.js v1.9.4 for the delivery map; Google Fonts CDN serving Cinzel, Playfair Display, and Poppins.
- Tooling: dotenv 17.2.2 for env configuration, Morgan 1.10.0 for request logging, body-parser 1.20.3 for HTTP parsing.



mongoDB



Leaflet



References



- Node.js Official Documentation – <https://nodejs.org/en/docs>
- Express.js Guide – <https://expressjs.com/en/guide/routing.html>
- MongoDB Documentation – <https://www.mongodb.com/docs>



- Mongoose ODM Documentation – <https://mongoosejs.com/docs>
- EJS Templating Documentation – <https://ejs.co/#docs>
- Leaflet.js Reference – <https://leafletjs.com/reference.html>
- JSON Web Token (JWT) RFC 7519 – <https://datatracker.ietf.org/doc/html/rfc7519>
- Indian Copyright Act, 1957 – Government of India, Ministry of Commerce.



THANK
YOU